

# AI Skills

Поглиблено. Автоматизація повсякденних фронтенд задач з ШІ

# План

- Як може бути корисним ШІ в програмуванні (на прикладі HTML, CSS, JS)?
- Як ШІ допомагає писати тести для JavaScript функцій?
- Які можливості має генерація тестів?
- Які недоліки у згенерованих тестах?
- Як автоматично генерувати документацію до коду (коментарі, README, пояснення)?
- Як пояснювати чужий HTML/CSS/JS код або код з реального проєкту?
- Використання ШІ для створення шаблонів сторінок, блоків, скриптів.
- Які складнощі можуть бути при генерації коду?
- Як вибір мови програмування впливає на якість генерації?
- Чи можна користуватися ШІ безкоштовно та яка різниця з платними версіями?

# Вступ. Як фронтенд-розробники можуть використовувати ШІ вже сьогодні

AI Skills

## Як може бути корисним ШІ в FrontEnd розробці?

### Що саме може робити ШІ для фронтенд-розробника:

- Генерувати базову HTML-розмітку сторінки (header, main, footer).
- Створювати блоки інтерфейсу: меню, картки товарів, галереї, форми.

### Писати CSS:

- сітки на Flexbox / Grid;
- адаптивну верстку під мобільні / планшети / десктопи;
- стилі для кнопок, карток, модальних вікон.

### Писати JavaScript для:

- роботи з DOM (показати/сховати елемент, змінити текст, додати/зняти клас);
- обробки подій (click, input, submit, keydown);
- прості валідації форм.

### Також ШІ може:

- пояснювати помилки з консолі;
- підказувати рефакторинг коду;
- допомагати вивчати нові можливості HTML/CSS/JS.

# Вступ. Як фронтенд-розробники можуть використовувати ШІ вже сьогодні

AI Skills

## Як ШІ допомагає писати тести?

### Згенерувати unit-тести для окремої функції:

- ти вставляєш функцію (наприклад, яка сортує масив або фільтрує дані);
- просиш: "Напиши unit-тести на JavaScript з console.assert"

### Автоматично побудувати структуру:

- Arrange – підготовка вхідних даних;
- Act – виклик функції;
- Assert – перевірка очікуваного результату.

### Підібрати виняткові кейси:

- порожні масиви, null/undefined;
- дуже довгі рядки;
- некоректні типи даних.

### Згенерувати мок-дані:

- масиви users, products, todos;
- різні сценарії для тестування фільтрації, сортування, пошуку.

# Вступ. Як фронтенд-розробники можуть використовувати ШІ вже сьогодні

AI Skills

## Які можливості має генерація тестів?

### Промпт:

"Напиши unit-тести для цієї функції на JavaScript, використовуючи console.assert. Ось мій код: ..."

### Відповідь:

```
console.assert(sum(2, 3) === 5, "2 + 3 має дорівнювати 5");  
console.assert(sum(-1, 1) === 0, "-1 + 1 має дорівнювати 0");  
console.assert(sum(0, 0) === 0, "0 + 0 має дорівнювати 0");
```

### Які можливості є в генерації тестів:

Автоматично створити декілька тестів з різними вхідними даними.

Сформулювати зрозумілі назви тестів (describe/it у Jest/Vitest).

Запропонувати додаткові edge cases:

- порожні значення;
- некоректні типи;
- дивні або рідкісні сценарії.

# Вступ. Як фронтенд-розробники можуть використовувати ШІ вже сьогодні

AI Skills

## Які недоліки у згенерованих тестах?

### Загальні назви тестів:

- типу "should work correctly" замість "повертає суму двох чисел".
- Не всі граничні випадки враховано:
- немає тестів на порожні значення, null, undefined, нестандартні дані.

### Помилки в логіці перевірки:

- неправильні очікувані значення;
- некоректні порівняння.

### Тести не відображають реальні бізнес-вимоги:

- ШІ бачить тільки код, але не знає контекст продукту.

### Можуть бути синтаксичні помилки: неправильні імпорти;

- забуті дужки;
- використання API, якого немає у вашому середовищі.

# Вступ. Як фронтенд-розробники можуть використовувати ШІ вже сьогодні

AI Skills

## За цільовим призначенням

### Пояснення до функцій:

```
// Обчислює суму двох чисел і повертає результат
function sum(a, b) {
  return a + b;
}
```

### Пояснення селекторів в CSS:

```
/* Остання картка в ряду – без правого відступу */
.card:last-child {
  margin-right: 0;
}
```

### Опис секції HTML:

```
<!-- Hero-блок із заголовком і кнопкою дії -->
<section class="hero">

...
</section>
```

### README.md для фронтенд-проєкту

ШІ може описати:

- структуру файлів (index.html, styles.css, script.js);
- як запустити проєкт (відкрити файл у браузері або використати Live Server).

Приклад промпта:

"Згенеруй README.md для маленького проєкту на HTML, CSS, JavaScript. Ось структура файлів: ..."

# Вступ. Як фронтенд-розробники можуть використовувати ШІ вже сьогодні

AI Skills

## Як пояснювати чужий код або код з реального проекту?



- Спирайся на документацію та імена. Бажано мати зрозумілий неймінг та написану документацію.
- Залучай тести (якщо є). Це допоможе зрозуміти поведінку коду.
- Використовуй аналогії або спрощення. Якщо логіка складна, поясни її простими словами.
- Розкажи: що, як і чому. Додайте подробиць які були задіяні технології та підходи.
- Розуміння контексту. Надавайте ШІ інформацію про проект і що наданий код робить.
- Поділ коду на логічні блоки. Розбийте код на зрозумілі частини.
- Коментарі до ключових рядків. Не соромтесь додавати у код додаткові пояснення.



# Вступ. Як фронтенд-розробники можуть використовувати ШІ вже сьогодні

AI Skills

## Використання ШІ для створення шаблонів

### HTML-шаблони:

- базова структура сторінки (doctype, head, body);
- каркас лендінгу: hero, features, testimonials, contact;
- форми (логін, реєстрація, підписка).

### CSS-шаблони:

- сітка на Flexbox або Grid;
- адаптивна верстка (mobile-first);
- стилі для кнопок, карток, модалок.

### JavaScript-шаблони:

- код для відкриття/закриття модального вікна;
- валідація полів форми перед відправкою;
- простий таб-інтерфейс (перемикання вкладок);
- показ/приховування мобільного меню.

### Приклади промптів:

"Створи простий landing page на HTML/CSS без фреймворків."

"Напиши JavaScript, який відкриває модальне вікно по кліку на кнопку 'Відкрити' і закриває по кліку на 'Закрити'."

# Вступ. Як фронтенд-розробники можуть використовувати ШІ вже сьогодні

AI Skills

## Які складнощі можуть бути при генерації коду?

### Потрібно перевіряти результат вручну:

- ШІ не бачить ваш реальний макет;
- не знає про всі обмеження проєкту (дизайн-гайд, браузерна підтримка).

### Зайві або нелогічні стилі:

- дублікати CSS правил;
- надто складні селектори;
- інлайн стилі замість окремого файлу.

### Недостатньо контексту:

- якщо не показати HTML, ШІ може написати CSS, який не підходить до вашої структури;
- якщо не показати частину JS, він може зробити зайві або конфліктні зміни.

### Складність промптів:

- якісний результат потребує:
- чіткої структури вимог,
- опису поведінки,
- вказівки, чи потрібна адаптивність, анімації тощо.

### Технічні уточнення:

- чи використовувати сучасний синтаксис (let/const, стрілкові функції);
- чи допустимі нові можливості CSS (grid, clamp, :has);
- які браузери треба підтримувати.

# Вступ. Як фронтенд-розробники можуть використовувати ШІ вже сьогодні

AI Skills

## Переваги і ризики використання ШІ в фронтенді?

### Переваги

- **Прискорення розробки:** генерація повторюваних блоків HTML, CSS, JS.
- **Підвищення продуктивності:** автодоповнення у редакторі; швидкі приклади рішень.
- **Підтримка навчання:** пояснення складних концепцій (event loop, flex, grid); показ альтернативних реалізацій.
- **Документація:** README, коментарі, пояснення структур сторінок.
- **Тестування:** генерація unit-тестів для JS-функцій;

### Ризики

- **Ненадійний або небезпечний код:** XSS, неправильна обробка введення; неефективні або застарілі патерни.
- **Втрата навичок:** залежність від підказок замість самостійного пошуку рішень.
- **Проблеми з авторським правом:** неясне походження фрагментів коду.
- **Галюцинації ШІ:** вигадані API або властивості; невірні пояснення.
- **Аспекти безпеки:** можливість згенерувати небезпечні конструкції без валідації.

# Вступ. Як фронтенд-розробники можуть використовувати ШІ вже сьогодні

AI Skills

## Якість генерації коду залежить від мови програмування?

### Від чого залежить:

- Обсяг тренувальних даних
- Популярність у спільноті
- Складність синтаксису
- Інтерфейси, DSL, шаблони

| Мова                            | Якість генерації | Коментар                                                |
|---------------------------------|------------------|---------------------------------------------------------|
| <b>Python</b>                   | Висока           | Багато прикладів, простий синтаксис                     |
| <b>JavaScript/TypeScript</b>    | Висока           | Часто оновлюється, багато прикладів                     |
| <b>C# / .NET</b>                | Добра            | Добре підтримується, але іноді бувають неточності з API |
| <b>Java</b>                     | Добра            | Чітка структура, але трохи багатослівна                 |
| <b>C++ / Rust</b>               | Змінна           | Більше шансів на помилки в шаблонах, типах              |
| <b>Go</b>                       | Добра            | Простий синтаксис, але менше гнучкості                  |
| <b>Haskell / Scala / Kotlin</b> | Середня/мінлива  | Може бути хороше пояснення, але код потребує перевірки  |

# Вступ. Як фронтенд-розробники можуть використовувати ШІ вже сьогодні

AI Skills

## Можна використовувати ШІ безкоштовно? Яка різниця у порівнянні з платними?

| Параметр    | Безкоштовна версія                            | Платна версія                                      |
|-------------|-----------------------------------------------|----------------------------------------------------|
| Модель      | GPT-3.5, Claude 1-2, базовий Gemini           | GPT-5, GPT-4.5, Claude 3, Gemini 1.5 Pro           |
| Якість коду | Середня: більше помилок, менше контексту      | Вища точність, більше коду в контексті             |
| Контекст    | ~4–8K токенів                                 | 32K–128K токенів (великі проєкти!)                 |
| Швидкість   | Повільніше, з можливими затримками            | Пріоритетна обробка                                |
| Функції     | Без доступу до плагінів, файлів, інструментів | Доступ до інструментів (код, зображення, веб, API) |
| Доступність | Може бути обмежено при високому навантаженні  | Завжди доступний                                   |

# Навчальний центр інформаційних технологій

**CyberBionic**  
*s y s t e m a t i c s*

